Visible Light Communication Systems with Microcontrollers or FPGAs

Wu Li Min[1] and Khong Yin Jou[2]

[1] Electrical and Computer System Engineering Discipline, School of Engineering, Monash University Malaysia

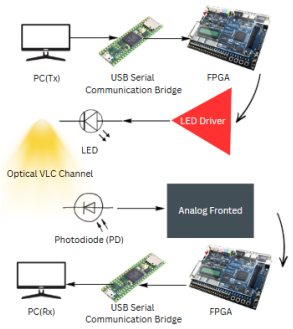
[2] Electrical and Computer System Engineering Discipline, School of Engineering, Monash University Malaysia

ABSTRACT

This research focuses on the design of a Serial Peripheral Interface (SPI) to improve PC–FPGA transmission speed in a Visible Light Communication (VLC) system. The current UART-based link limits throughput and constrains overall system performance. A Teensy 4.1 microcontroller was introduced between the PC and FPGA to replace UART with synchronous, full-duplex SPI using dedicated clock (SCLK) and data (MOSI/MISO) lines. The SPI interface was implemented in VHDL on the FPGA , with the Teensy 4.1 programmed in C++ to realise the master and slave logic. Oscilloscope measurements of the Teensy as SPI master configuration show one 8-byte packet transferred over approximately 14 µs, giving an effective throughput of approximately 571 kB/s (4.57 Mbps) within the packet. The FPGA as master configuration transferred a 14-byte (112-bit) packet in 1.128 ms, corresponding to approximately 99.3 kbps (12.42 kB/s). These results show that the SPI-based interface offers a substantial improvement in the PC–FPGA communication speed compared with the previous UART-based implementation.

KEYWORDS: Serial Peripheral Interface (SPI), UART, Visible Light Communication (VLC), Data Transmission Speed

1. INTRODUCTION

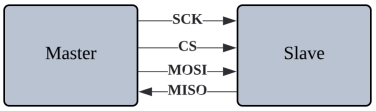


(Figure 1. Visible Light Communication (VLC) system)

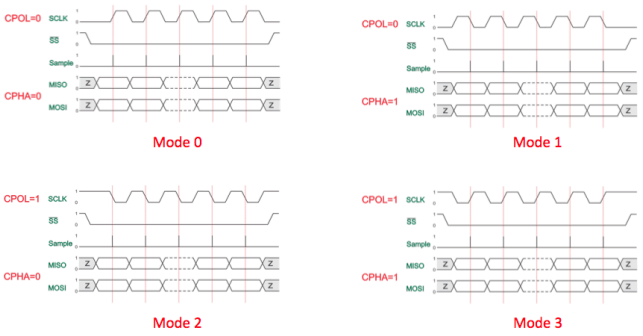
In the current VLC setup used in this project, the optical channel achieves a data rate of approximately 50 kB/s. However, the communication link between the PC and the FPGA is currently implemented using the Universal Asynchronous Receiver/Transmitter (UART) protocol, which is limited to approximately 14.4 kB/s. This creates a significant bottleneck, as the optical channel itself is capable of much higher throughput while the overall system performance is constrained by the slower PC–FPGA interface, so the capability of the optical channel can never be fully utilised. Poorani and Kurunjimalar (2016) implemented UART and SPI on a single FPGA. They showed that SPI's synchronous clocking remains reliable even with imprecise clock rates, unlike UART's dependence on accurate baud timing, while also

reducing processor count and system cost. Therefore, the objective of this research is to replace the UART based PC–FPGA link with an SPI based architecture, using a Teensy 4.1 as an intermediary between the PC and FPGA. Specifically, this project aims to design and simulate SPI Master/Slave modules in VHDL (Quartus) and C++ (PlatformIO), implement full-duplex synchronous data transfer between the PC, Teensy 4.1, and DE2-115 FPGA. This report documents the methodology, results, and discussion of findings.

2. METHODOLOGY



(Figure 2. Master and Slave system)



(Figure 3. 4 SPI Modes)

The proposed system is represented in **Figure 2**, comprising a Master and Slave device connected via SPI. The clock signal, generated by the Master, controls data transfer timing in both directions. The Chip Select (CS) line indicates whether the Slave is active: CS high means no communication occurs, while CS low selects the device for communication. The MOSI (Master Out, Slave In) line carries data from Master to Slave, and the MISO (Master In, Slave Out) line carries data from Slave to Master. SPI also defines four clock modes (Mode 0 – 3), based on Clock Polarity (CPOL) and Clock Phase (CPHA), which set the clock's idle state and the edge on which data is sampled, as shown in **Figure 3**. This design uses Mode 0 (CPOL=0, CPHA=0), where the clock idles low and data is sampled on the rising edge. Two scenarios were used to demonstrate this system.

A. Teensy 4.1 as Master, FPGA as Slave

ASCII codes are sent from Teensy 4.1 to the FPGA over SCK, CS, and MOSI (Mode 0) for display on 7-segment digits. Up to 8 characters are read and transmitted individually as ASCII bytes, each with its own chip select pulse. The FPGA synchronizes the SPI signals and shifts in each byte via a 3-state Finite state machine (FSM) (IDLE → RECEIVE → TIMEOUT), latching completed bytes into one of eight registers and clearing incomplete data on timeout. Each latched byte is then decoded by a BCD module into 7-segment output, displaying the corresponding character on digit board.

B. FPGA as Master, Teensy 4.1 as Slave

The FPGA reads the 16 switches as 2 bytes of input and, upon a button press, transmits a 14 bytes packet (Table 1) over SCK, CS, and MOSI (Mode 0). On the receiving end, Teensy 4.1 uses the external SPI_MSTransfer_T4 library to receive the packet through a callback

function, decoding the switch data into two ASCII characters. Both the characters and the binary value of each byte are then displayed on the Serial Monitor for verification.

Table 1. Composition of the packet

Word	Content	Size	Word	Content	Size
0	Header (fixed value)	2 bytes	4	Packet ID (increases each send)	2 bytes
1	Slave ID (must match Teensy)	2 bytes	5	Switch data (payload)	2 bytes
2	Length field	2 bytes	6	Checksum (Packet ID XOR Switch data)	2 bytes
3	Command code	2 bytes			

3. RESULTS & DISCUSSIONS

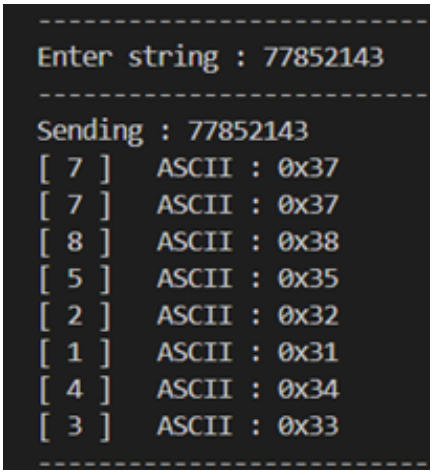
A. Teensy 4.1 as Master and FPGA as Slave



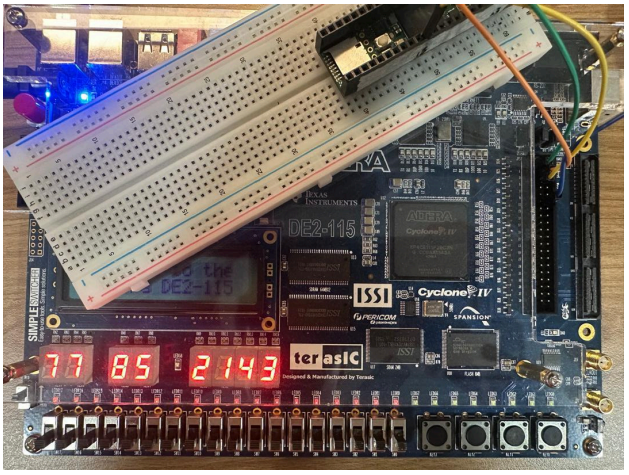
(Figure 4. Multiple-byte SPI waveform)



(Figure 5. Single-byte SPI waveform)



(Figure 6. Input of Master)

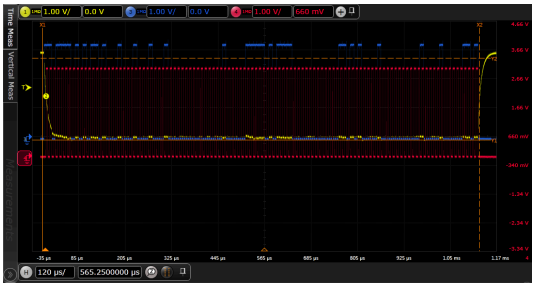


(Figure 7. Output of Slave)

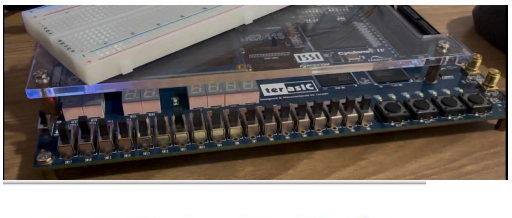
As shown in **Figure 6**, the string "77852143" entered on the Teensy was correctly converted to ASCII hex values (e.g., '7' → 0x37, '8' → 0x38) and transmitted byte-by-byte over SPI. The

FPGA correctly received and decoded each byte, displaying "77 85 21 43" on the 7-segment digits, matching the transmitted string exactly. This confirms correct synchronization, byte latching, and BCD decoding on the FPGA side with no data loss observed. Oscilloscope capture of a single SPI transaction shows the chip-select line held low for approximately 14 µs, during which 64 clock cycles transfer one 8 byte packet. This corresponds to an effective throughput of approximately 571 kB/s (4.57 Mbps) within the packet.

B. FPGA as Master and Teensy 4.1 as Slave



(Figure 8. 14-byte packet SPI waveform)



(Figure 9. Input/Output of Master)

The FPGA read the 16 switches as 2 bytes (on = 1, off = 0) and transmitted a 14 byte packet upon a button press. As shown in **Figure 9**, Teensy 4.1 correctly received and decoded the packet, displaying "B." and its binary value "01000010 00000000" on the Serial Monitor, matching the switch configuration. This confirms correct packet parsing and successful full-duplex communication in the FPGA-as-Master configuration. A complete 14 byte (112 bits) packet was captured with the chip-select line asserted throughout, spanning Δt = 1.128 ms. This corresponds to a bit rate of approximately 12.42 kB/s (99.3 kbps) .

4. CONCLUSIONS

This project successfully replaced UART with SPI for the PC–FPGA link, using a Teensy 4.1 to enable communication. In the Teensy as Master configuration, oscilloscope measurements confirmed an SPI clock frequency of approximately 5 MHz, corresponding to a raw data rate of approximately 571 kB/s (4.57 Mbps) . The FPGA successfully received and decoded all eight transmitted ASCII bytes without observable data loss. In the FPGA as Master configuration, a transmission rate of approximately 12.42 kB/s (99.3 kbps) was achieved. Both configurations demonstrate reliable bidirectional transfer, with the throughput difference reflecting the lower clock rate used on the FPGA side master. Further improvement may be achieved by reducing chip-select and overhead between bytes and by raising the SPI clock frequency while maintaining reliable communication.

5. REFERENCES

E-Tinkers. 2020. Do you know Arduino? – SPI and Arduino SPI Library. *E-Tinkers.com*. https://e-tinkers.com/2020/03/do-you-know-arduino-spi-and-arduino-spi-library/

Poorani, M. & Kurunjimalar, R. 2016. Design implementation of UART and SPI in single FPGA. *2016* 10th International Conference on Intelligent Systems and Control (ISCO), Coimbatore, India.

tonton81. n.d. SPISlave_T4 [GitHub repository]. https://github.com/tonton81/SPISlave_T4